

《概论》第 11 章、12 章讨论事务处理技术。事务处理技术主要包括数据库恢复技术和并发控制技术，它们是数据库管理系统的重要组成部分。本章讨论数据库恢复的概念和常用技术。

11.1 基本知识点

① 需要了解的：了解数据库运行过程中可能产生的故障类型，它们如何影响事务的正常执行、如何破坏数据库数据；了解数据转储的概念及分类；了解什么是数据库镜像功能。

② 需要牢固掌握的：掌握事务的基本概念和事务的 ACID 特性；掌握数据库恢复的实现技术；掌握日志文件的内容及作用、登记日志文件所要遵循的原则；掌握具有检查点的恢复技术。

③ 需要举一反三的：恢复的基本原理，针对不同故障的恢复策略和方法。

④ 难点：日志文件的使用、系统故障的恢复策略，以及具有检查点的恢复技术。

事务管理模块是 DBMS 实现中的关键技术。事务恢复的基本原理是冗余，它貌似简单，真正实现的过程却很复杂。数据库的事务管理策略（不仅有数据库恢复策略，还有并发控制策略）和 DBMS 缓冲区管理策略、事务隔离级别等密切相关。

读者要掌握数据库故障恢复的策略和方法。对于刚刚学习数据库课程的读者来讲，或许并不能体会数据库故障恢复的复杂性和重要性。到了实际工作中，作为数据库管理员，则必须十分清楚每一个使用的 DBMS 产品提供的恢复技术以及恢复方法，并且能够根据这些技术正确制定出实际系统的恢复策略，以保证数据库系统 7×24 小时正确运行，保证数据库系统在遇到故障能及时恢复正常运行，提高系统抗灾难的能力。

11.2 习题解答和解析

1. 试述事务的概念及事务的 4 个特性。数据库恢复技术能保证事务的哪些特性？

【解析】请阅读《概论》第 11 章 11.1 节“事务的基本概念”相关内容。

事务是用户定义的一个数据库操作序列，这些操作要么全做，要么全不做，是一个不可分割的工作单位。

事务具有 4 个特性：原子性（atomicity）、一致性（consistency）、隔离性（isolation）和持

续性(durability)。这4个特性通常简称为ACID特性。

原子性：事务是数据库的逻辑工作单位，事务中包括的诸操作要么都做，要么都不做。

一致性：事务执行的结果必须是使数据库从一个一致性状态转为另一个一致性状态。

隔离性：一个事务的执行不能被其他事务干扰，即一个事务的内部操作及使用的数据对其他并发事务是隔离的，并发执行的各个事务之间不能互相干扰。

持续性：持续性也称永久性(permanence)，指一个事务一旦提交，它对数据库中数据的改变就应该是永久性的，接下来的其他操作或故障不应该对其执行结果有任何影响。

数据库恢复可以保证事务的原子性与持续性。

为了保证事务的隔离性和一致性，数据库管理系统需要对并发操作进行控制，第12章将进一步讲解并发控制。

2. 为什么事务非正常结束会影响数据库数据的正确性？请举例说明之。

【解析】事务执行的结果必须是使数据库从一个一致性状态转为另一个一致性状态。如果在数据库系统运行中发生故障，有些事务尚未完成就被迫中断，这些未完成的事务对数据库所做的修改有一部分已写入物理数据库，这时数据库就处于一种不正确的状态，或者说的不一致的状态。

例如，某工厂的库存管理系统中，要把数量为 Q 的某种零件从仓库1移到仓库2存放，则可以定义一个事务 T ， T 包括两个操作： $Q1=Q1-Q$ ， $Q2=Q2+Q$ 。如果 T 非正常终止时只做了第一个操作，则数据库就处于不一致性状态，库存量无缘无故少了 Q 。

3. 登记日志文件时为什么必须先写日志文件，后写数据库？

【解析】请阅读《概论》第11章11.4.2小节“登记日志文件”相关内容。

把对数据的修改写到数据库中和把表示这个修改的日志记录写到日志文件中，是两个不同的操作。有可能会在这两个操作之间发生故障，即这两个写操作只完成了一个。

如果先写了数据库修改，而在运行记录中没有登记这个修改，则以后就无法恢复这个修改了。如果先写日志，但没有修改数据库，那么在恢复时只不过是多执行一次UNDO操作，并不会影响数据库的正确性。所以一定要先写日志文件，即首先把日志记录写到日志文件中，然后再写数据库的修改。

4. 考虑下表所示的日志记录：

序号	日志
1	T_1 : 开始
2	T_1 : 写 A , $A=10$
3	T_2 : 开始
4	T_2 : 写 B , $B=9$
5	T_1 : 写 C , $C=11$

续表

序号	日志
6	T_1 : 提交
7	T_2 : 写 C , $C=13$
8	T_3 : 开始
9	T_3 : 写 A , $A=8$
10	T_2 : 回滚
11	T_3 : 写 B , $B=7$
12	T_4 : 开始
13	T_3 : 提交
14	T_4 : 写 C , $C=12$

- ① 如果系统故障发生在序号 14 之后, 说明哪些事务需要重做, 哪些事务需要回滚。
- ② 如果系统故障发生在序号 10 之后, 说明哪些事务需要重做, 哪些事务需要回滚。
- ③ 如果系统故障发生在序号 9 之后, 说明哪些事务需要重做, 哪些事务需要回滚。
- ④ 如果系统故障发生在序号 7 之后, 说明哪些事务需要重做, 哪些事务需要回滚。

答:

- ① 重做: T_1 、 T_3 ; 回滚: T_2 、 T_4 。

因为 T_1 和 T_3 事务已经提交, 所以重做; T_2 回滚, T_4 执行中, 没有提交, 所以要回滚。

- ② 重做: T_1 ; 回滚: T_2 、 T_3 。

因为 T_1 事务已经提交, 所以重做; T_2 回滚, T_3 执行中, 没有提交, 所以要回滚。 T_4 事务还没有开始。

- ③ 重做: T_1 ; 回滚: T_2 、 T_3 。

- ④ 重做: T_1 ; 回滚: T_2 。

5. 考虑第 4 题所示的日志记录, 假设开始时 A 、 B 、 C 的值都是 0:

- ① 如果系统故障发生在序号 14 之后, 写出系统恢复后 A 、 B 、 C 的值。
- ② 如果系统故障发生在序号 12 之后, 写出系统恢复后 A 、 B 、 C 的值。
- ③ 如果系统故障发生在序号 10 之后, 写出系统恢复后 A 、 B 、 C 的值。
- ④ 如果系统故障发生在序号 9 之后, 写出系统恢复后 A 、 B 、 C 的值。
- ⑤ 如果系统故障发生在序号 7 之后, 写出系统恢复后 A 、 B 、 C 的值。
- ⑥ 如果系统故障发生在序号 5 之后, 写出系统恢复后 A 、 B 、 C 的值。

答:

- ① $A=8, B=7, C=11$ 。
- ② $A=10, B=0, C=11$ 。
- ③ $A=10, B=0, C=11$ 。
- ④ $A=10, B=0, C=11$ 。
- ⑤ $A=10, B=0, C=11$ 。
- ⑥ $A=0, B=0, C=0$ 。

6. 针对不同的故障类型(事务故障、系统故障、介质故障), 试给出恢复的策略和方法。

【解析】请阅读《概论》第11章11.5节“恢复策略”相关内容。

事务故障的恢复步骤是:

- ① 反向扫描日志文件, 查找该事务的更新操作。
 - ② 对该事务的更新操作执行逆操作, 即将日志记录中“更新前的值”写入数据库。
- 直至读到此事务的开始标记, 该事务故障的恢复就完成了。

系统故障的恢复步骤是:

- ① 正向扫描日志文件, 找出在故障发生前已经提交的事务, 将其事务标识记入重做队列(REDO-LIST); 同时找出故障发生时尚未完成的事务, 将其事务标识记入撤销队列(UNDO-LIST)。
- ② 对撤销队列中的各个事务进行 UNDO 处理。
- ③ 对重做队列中的各个事务进行 REDO 处理。

介质故障的恢复步骤是:

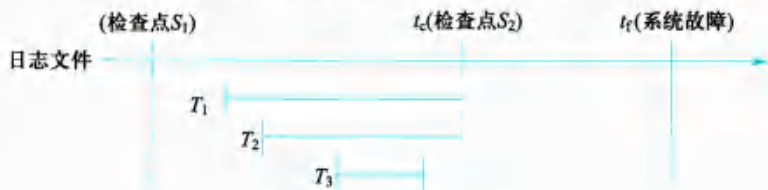
- ① 装入最新的数据库后备副本(离故障发生时刻最近的转储副本), 使数据库恢复到最近一次转储时的一致性状态。
- ② 装入转储结束时刻的日志文件副本。
- ③ 启动系统恢复命令, 由 DBMS 完成恢复功能, 即重做已完成的事务。

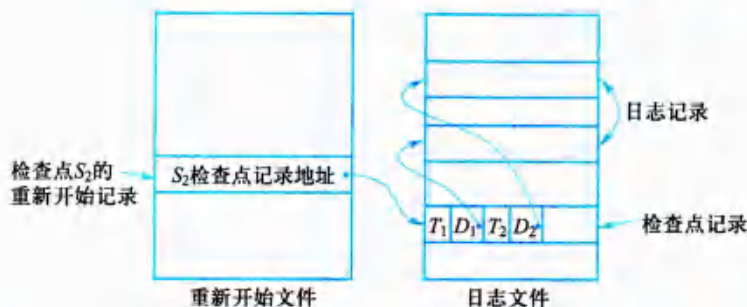
7. 什么是检查点记录? 检查点记录包括哪些内容?

【解析】请阅读《概论》第11章11.6节“具有检查点的恢复技术”相关内容。

检查点记录是一类新的日志记录。它的内容包括:

- ① 建立检查点时刻所有正在执行的事务清单, 如图中的 T_1 、 T_2 。
- ② 这些事务的最近一个日志记录的地址, 如图中的 D_1 、 D_2 。



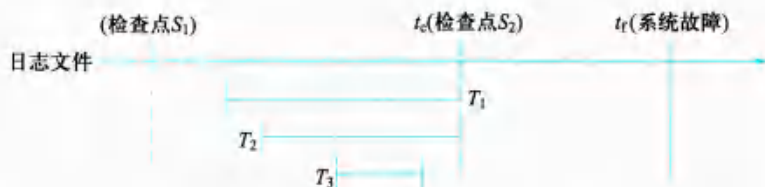


8. 具有检查点的恢复技术有什么优点？试举一个具体的例子加以说明。

答：利用日志技术进行数据库恢复时，恢复子系统必须搜索整个日志，这将耗费大量的时间。此外，需要 REDO 处理的事务实际上已将其更新操作结果写到数据库中了，然而恢复子系统又重新执行了这些操作，浪费了大量时间。

检查点技术就是为了解决这些问题。

例如：



在采用检查点技术之前，恢复时需要从头扫描日志文件，而利用检查点技术只需要从 t_c 开始扫描日志，这就缩短了扫描日志的时间。

事务 T_3 的更新操作实际上已经写到数据库中了，进行恢复时没有必要再 REDO 处理，采用检查点技术做到了这一点。

9. 试述使用检查点方法进行恢复的步骤。

答：

① 在重新开始文件（见第 7 题的图）中，找到最后一个检查点记录在日志文件中的地址，由该地址在日志文件中找到最后一个检查点记录。

② 由该检查点记录得到检查点建立时刻所有正在执行的事务清单 ACTIVE-LIST。

这里建立两个事务队列：UNDO-LIST 为需要执行 UNDO 操作的事务集合；REDO-LIST 为需要执行 REDO 操作的事务集合。

把 ACTIVE-LIST 暂时放入 UNDO-LIST 队列，REDO-LIST 队列暂为空。

③ 从检查点开始正向扫描日志文件。

如有新开始的事务 T_i ，把 T_i 暂时放入 UNDO-LIST 队列。

如有已提交的事务 T_j , 把 T_j 从 UNDO-LIST 队列移到 REDO-LIST 队列, 直到日志文件结束。

④ 对 UNDO-LIST 中的每个事务执行 UNDO 操作, 对 REDO-LIST 中的每个事务执行 REDO 操作。

10. 什么是数据库镜像? 它有什么用途?

【解析】请阅读《概论》第11章 11.7 节“数据库镜像”相关内容。

数据库镜像即根据 DBA 的要求, 自动把整个数据库或其中的关键数据复制到另一个磁盘上。每当主数据库更新时, DBMS 自动把更新后的数据复制过去, 即 DBMS 自动保证镜像数据与主数据库的一致性。

数据库镜像的用途有:

① 用于数据库恢复。当出现介质故障时, 镜像磁盘可继续提供使用, 同时 DBMS 自动利用镜像磁盘数据进行数据库的恢复, 不必关闭系统和重装数据库副本。

② 提高数据库的可用性。在没有出现故障时, 当一个用户对数据加排他型锁进行修改时, 其他用户可以读镜像数据库上的数据, 而不必等待该用户释放锁。

11.3 补充习题

1. 问答题

① 在系统故障的恢复策略中, 为什么 UNDO 处理反向扫描日志文件, REDO 处理正向扫描日志文件?

② 说明恢复系统是否可以保证事务的原子性和持续性。

2. 综合题

考虑下表所示的日志记录:

序号	日志
1	T_1 : 开始
2	T_1 : 写 A
3	T_2 : 开始
4	T_2 : 写 B
5	T_3 : 开始
6	T_1 : 提交
7	T_2 : 回滚